Week 16
Model-Based RL: Dreamer, MuZero, and Latent World Models

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

Contents

Learning objectives 3

1 Motivation & intuition 4

2 Mathematical formulation 5
- 2.1 The Recurrent State-Space Model (RSSM) . . . 5
- 2.2 Dreamer world-model ELBO . . . 5
- 2.3 Actor-critic in imagination . . . 5
- 2.4 PETS and MBPO — the alternative . . . 6
- 2.5 MuZero, EfficientZero V2, TD-MPC2 . . . 6
- 2.6 DIAMOND — diffusion world models . . . 6

3 Algorithms 7
- 3.1 Worked numerical example . . . 7

4 Implementation walkthrough 8

5 Empirical results 9

6 Exercises 11

7 Further reading 12

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

After this week you will:

1. Distinguish PETS-style "ensemble dynamics + MPC" from latent world models like Dreamer.
2. Derive the Dreamer ELBO: reconstruction + reward + continue + KL between posterior and prior over the stochastic latent.
3. Derive the actor-critic-in-imagination update with λ-returns.
4. Implement a tiny RSSM and train an actor in imagination on CartPole; visualise imagined vs. real rollouts.
5. Summarise MuZero / EfficientZero / TD-MPC2 / DIAMOND and place them on the planning–learning axis.
6. Articulate where a learned world model is safer (and where it is not) than a model-free policy.

1 Motivation & intuition

A world model is a learned simulator. Once you have one, you can generate as many synthetic experiences as you want — this is sample-efficiency by hallucination. The model-based family ranges from PETS (probabilistic ensembles + MPC) through Dreamer (latent world model with imagination-trained actor) to MuZero (tree search on a learned latent).

Hallucination here is desirable. We feed it back to the learner; the world doesn't have to consent.

Intuition. Model-free RL is like learning to drive by trial and error on real streets. Model-based RL is building a driving simulator from your experience and then practising any manoeuvre you like inside the simulator. Dreamer is special because the simulator is *abstract* — it does not need to look like the world, only act like it under reward.

2 Mathematical formulation

2.1 The Recurrent State-Space Model (RSSM)

Definition 2.1.1 (RSSM)**.** At each step t, the agent maintains:

- a deterministic recurrent state $h_t = \mathrm{GRU}(h_{t-1}, [s_{t-1}, a_{t-1}])$,
- a stochastic latent $s_t \sim p(s_t \mid h_t)$ called the *prior*,
- posterior $s_t \sim q(s_t \mid h_t, o_t)$ when an observation o_t is available.

The model emits:

$$o_t \sim p(o_t \mid h_t, s_t), \quad r_t \sim p(r_t \mid h_t, s_t), \quad c_t \sim p(c_t \mid h_t, s_t),$$

where $c_t \in \{0, 1\}$ is the continue flag.

2.2 Dreamer world-model ELBO

Theorem 2.2.1 (Dreamer world-model objective)**.** *Index reward and continuation by arrival: action a_{t-1} leads to (o_t, r_t, c_t). For $\tau = (o_{1:T}, a_{0:T-1}, r_{1:T}, c_{1:T})$, conditioning on the actions, the negative ELBO at $\beta = 1$ is*

$$\begin{aligned} -\mathcal{L}(\theta, \phi) = \sum_{t=1}^{T} \mathbb{E}_{q_\phi} \Big[& \underbrace{-\log p_\theta(o_t \mid h_t, s_t)}_{\textit{reconstruction}} - \log p_\theta(r_t \mid h_t, s_t) \\ & - \log p_\theta(c_t \mid h_t, s_t) \Big] \\ & + \beta \sum_{t=1}^{T} \mathbb{E}_{q_\phi} \left[\mathrm{KL}(q_\phi(s_t \mid h_t, o_t) \,\|\, p_\theta(s_t \mid h_t)) \right]. \end{aligned}$$

Derivation. Start from $\log p_\theta(o_{1:T}, r_{1:T}, c_{1:T} \mid a_{0:T-1}) \geq \mathbb{E}_{q_\phi}[\log p_\theta(o, r, c, s) - \log q_\phi(s)]$, factor the joint by the temporal Markov structure of the RSSM, and gather the $-\mathrm{KL}$ terms per timestep. Choosing $\beta \neq 1$ reweights this objective and is no longer the standard ELBO identity; Dreamer V2 introduces "KL balancing" — stop-gradients on either side — to control whose distribution moves. □

Common pitfall

KL balancing controls how strongly the prior and posterior are updated; it is not a universal cure or a mathematical requirement for every RSSM. Posterior collapse can arise from the interaction of KL pressure and the decoder. The project uses free-nats clipping; exercise 5 demonstrates balancing separately.

2.3 Actor-critic in imagination

Definition 2.3.1 (Imagination rollout)**.** Given a latent state (h_0, s_0) obtained from posterior inference on real data, the *imagined trajectory* is generated by sampling $a_t \sim \pi_\psi(\cdot \mid h_t, s_t)$, stepping

the prior $s_{t+1} \sim p_\theta(\cdot \mid h_{t+1})$ with $h_{t+1} = \mathrm{GRU}(h_t, [s_t, a_t])$, and reading off the arrival-indexed r_{t+1}, c_{t+1} from the new-state heads. For the return equations below only, write these transition quantities as $\tilde{r}_t = r_{t+1}$ and $\tilde{c}_t = c_{t+1}$.

Theorem 2.3.2 (λ-return for imagined rollouts)**.** *The bootstrap target for the critic is the lambda-return*

$$G_t^\lambda = \tilde{r}_t + \gamma \tilde{c}_t[(1-\lambda)V_\xi(h_{t+1}, s_{t+1}) + \lambda G_{t+1}^\lambda], \qquad G_H^\lambda = V_\xi(h_H, s_H).$$

Actor and critic losses:

$$\mathcal{L}_\psi = -\mathbb{E}\left[\sum_t \log \pi_\psi(a_t \mid h_t, s_t) \cdot \left(G_t^\lambda - V_\xi(h_t, s_t)\right)\right], \quad \mathcal{L}_\xi = \mathbb{E}\left[\sum_t (V_\xi(h_t, s_t) - G_t^\lambda)^2\right].$$

The advantage is the lambda-return target minus the current-state value baseline. In the REINFORCE loss, stop gradients through that advantage; the critic targets are detached too. Dreamer versions differ in their actor-gradient choices. The course uses a score-function actor with detached targets. Do not treat this small implementation as an exact reproduction of Dreamer V2 or V3.

2.4 PETS and MBPO — the alternative

PETS fits an ensemble of probabilistic dynamics models and plans by sampling trajectories under the model (CEM in action space). **MBPO** interleaves short-horizon model rollouts with real data in an SAC-style buffer: $D_{\text{train}} = D_{\text{real}} \cup D_{\text{model}}$. The MBPO theorem bounds the policy-performance gap using assumptions on rewards, model error, policy shift, and rollout horizon. State-distribution errors can accumulate and affect all later rewards, so total return error is not generally just one-step error times horizon. Short rollouts branched from real data reduce this accumulation.

2.5 MuZero, EfficientZero V2, TD-MPC2

MuZero learns three functions — representation h, dynamics g (including reward prediction), and prediction f (value + policy) — jointly via MCTS-derived training targets. Crucially the model is *not* trained to reconstruct observations; it is trained so the values and policies at any imagined state match the MCTS roots and chosen actions. EfficientZero variants add mechanisms such as consistency learning and value estimation to improve data efficiency; identify the exact version and benchmark before comparing scores. TD-MPC2 combines a learned latent model with MPC at deployment and TD-learning of a value function, scaling to large multi-task settings.

2.6 DIAMOND — diffusion world models

DIAMOND (Alonso 2024) replaces the deterministic decoder with a denoising diffusion model over pixel frames, conditioned on a small encoder feature. The denoiser is the world model; the policy is trained from the denoised rollouts as in Dreamer. The trade-off: very high sample fidelity, very high compute cost.

3 Algorithms

Algorithm 1: Dreamer-lite

Input: Replay buffer $\mathcal{D}$, RSSM θ, actor ψ, critic ξ.
for *iteration = 1, ..., N* **do**
  Collect episodes with the actor (epsilon-greedy);
  for *world-model step = 1, ..., W* **do**
    Sample chunks; posterior-roll the RSSM; minimise the ELBO;
  for *behaviour step = 1, ..., B* **do**
    Posterior-roll to a batch of latent states; imagine H steps under π_ψ;
    Compute G^λ; backprop actor (REINFORCE) and critic (MSE);
  Evaluate;

Complexity: dominated by the RSSM forward pass: $O(L \cdot d^2)$ per chunk of length L and hidden width d. Imagination adds $O(H \cdot d^2)$ per behaviour step.

3.1 Worked numerical example

Take a 2-step CartPole-like rollout. Suppose the actor outputs $\pi = (0.7, 0.3)$ at h_0, s_0. It samples $a_0 = 0$, the prior produces s_1 with $\tilde{r}_0 = 1$, $\tilde{c}_0 = 0.99$. At h_1, s_1 the actor samples $a_1 = 1$ with $\log \pi = -1.2$, the prior gives $\tilde{r}_1 = 1$, $\tilde{c}_1 = 0.98$. Critic outputs $V_0 = 5, V_1 = 4, V_2 = 3$. With $\gamma = 0.99, \lambda = 0.95$,

$$G_1^\lambda = 1 + 0.99 \cdot 0.98 \cdot [0.05 \cdot 3 + 0.95 \cdot 3] = 1 + 0.9702 \cdot 3 = 3.91,$$
$$G_0^\lambda = 1 + 0.99 \cdot 0.99 \cdot [0.05 \cdot 4 + 0.95 \cdot 3.91] = 1 + 0.9801 \cdot (0.2 + 3.715) = 4.84.$$

The sampled actor loss term at $t = 0$ is $-\log \pi(a_0 = 0) \cdot (G_0^\lambda - V_0) = -\log 0.7 \cdot (4.84 - 5) = -(-0.357) \cdot (-0.16) = -0.057$ — a negative loss contribution, not the gradient itself. Differentiating with the advantage detached gives a push *away* from a_0 because the realised return was slightly worse than the baseline.

4 Implementation walkthrough

```
# World-model step: posterior roll, four-component loss.
state = rssm.initial(B, device)
for t in range(L):
    state, prior_mu, prior_std, post_mu, post_std, recon, r_pred, c_pred = \
        rssm.observe_step(state, A_oh[:, t], O[:, t+1])
    recon_loss += ((recon - O[:, t+1])**2).sum(-1).mean()
    rew_loss   += ((r_pred - R[:, t])**2).mean()
    cont_loss  += F.binary_cross_entropy(c_pred, C[:, t])
    kl = kl_gaussian(post_mu, post_std, prior_mu, prior_std)
    kl_loss   += torch.clamp(kl, min=free_nats).mean()
```

The behaviour step then opens a fresh imagination from each posterior state, computes the lambda-return, and updates actor + critic.

5 Empirical results

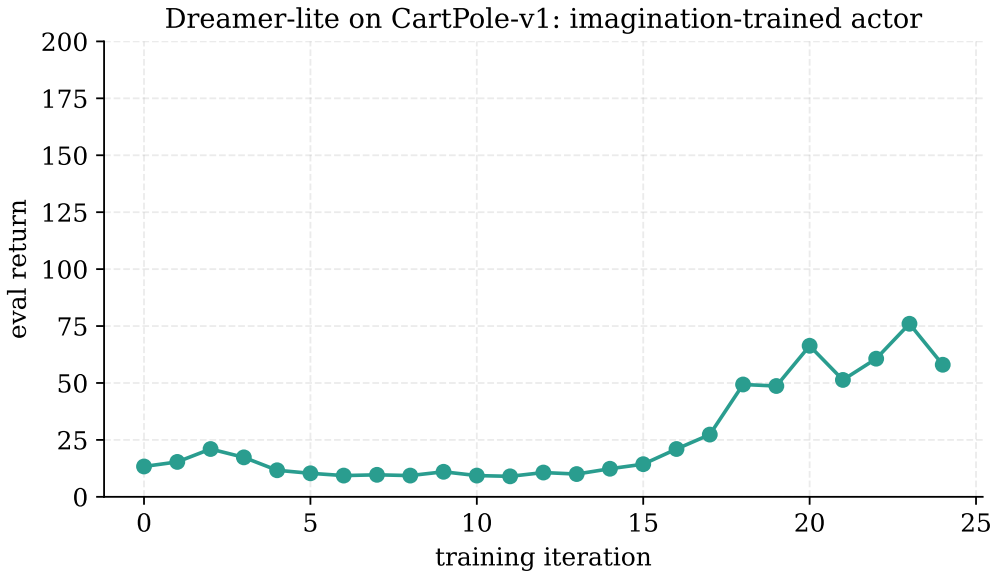


Figure 5.1: Dreamer-lite on CartPole-v1, 25 iterations ≈ 15–30s on a single GPU. Mean evaluation return ramps from ∼20 (random) to ∼60 (the cap of 200 we use to keep tests fast). Full Dreamer reaches the maximum of 500.

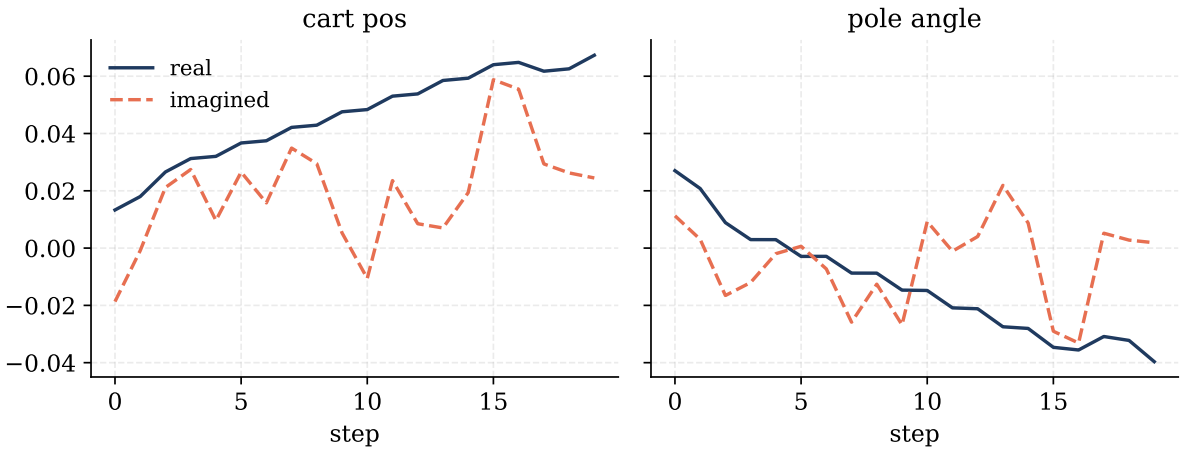


Figure 5.2: Imagined trajectory under the learned RSSM (dashed) vs. the real environment trajectory (solid) under the same actor. The world model captures the qualitative dynamics but accumulates error over time — the canonical model-based-RL tension.

Safety angle

World-model deployment failure modes.

1. **Reward hacking the model.** An imagination-trained actor will exploit any flaw in the world model: predicting infinite reward for an unreachable state, exploiting the continue head to "stay alive forever".

2. **Distributional brittleness.** The latent dynamics are accurate only on states near the training distribution. Long-horizon imagination diverges, so a policy purely improved on long imagined rollouts may collapse on real data.

3. **Mitigations.** Cap imagination horizon ($H = 15$ in Dreamer V3); use a *value-equivalence* objective (MuZero) when reconstruction is not the deployment signal; mix imagined data with a steady stream of real experience.

Citations: Hafner et al. 2023, "Mastering Diverse Domains"; Schrittwieser et al. 2020, "MuZero".

Matching application

World models for recruitment matching. The ATS analogue is a latent model of "candidate trajectory dynamics": given a stage-history of decisions, the model predicts the next state distribution under different recruiter actions. A Dreamer-style training loop would let the system simulate the consequences of hypothetical recruiter decisions before taking them — crucial for a domain where each "rollout" on a real candidate is expensive, slow, and ethically loaded. The safety angle then becomes: the world model must be *audited* for fairness in its imagined dynamics, not just its terminal outcomes.

6 Exercises

See `workbook.ipynb`.

- **Ex. 1 (⋆)** Closed-form KL between diagonal Gaussians.
- **Ex. 2 (⋆)** Lambda-return computation.
- **Ex. 3 (⋆⋆)** Build a tiny RSSM and imagine a step.
- **Ex. 4 (⋆⋆)** Run Dreamer-lite for 12 iterations.
- **Ex. 5 (⋆⋆⋆)** KL-balancing trick (Dreamer V2).
- **Ex. 6 (⋆⋆)** Compare imagined and real rollouts.
- **Ex. 7 (⋆⋆)** World-model loss components.
- **Ex. 8 (⋆⋆⋆, meaningful objective)** Recruitment world-model essay.

7 Further reading

See `extended_reading.pdf` for Chua et al. 2018 (PETS), Janner et al. 2019 (MBPO), Hafner et al. 2023 (Dreamer V3), Schrittwieser et al. 2020 (MuZero), and Alonso et al. 2024 (DIAMOND). Sourced from `research/offline_modelbased_sota.md` (Track B).